

15

Adding Persistence to Socket.IO Using MongoDB

Now that we have implemented a Socket.IO backend and frontend, let's spend some time integrating it with the MongoDB database by temporarily storing messages in the database and replaying them when a new user joins, so that users can see the chat history after they join. Additionally, we will refactor our chat app to be ready for future expansions and maintenance. Finally, we will test out the new structure by implementing new commands to join and switch rooms.

In this chapter, we are going to cover the following main topics:

- Storing and replaying messages using MongoDB
- Refactoring the app to be more extensible
- Implementing commands to join and switch rooms

Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing for Full Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1 and 2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch15>.

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

The CiA video for this chapter can be found at:

https://youtu.be/Mi7Wj_jxjHM.

Storing and replaying messages using MongoDB

Currently, if a new user joins the chat, they will not see any messages until someone actively sends messages. As such, new users will not be able to participate well in ongoing discussions. To solve this issue, we can store messages in the database and replay them when a user joins.

Creating the Mongoose schema

Follow these steps to create a Mongoose schema for storing chat messages:

1. Copy the existing **ch14** folder to a new **ch15** folder, as follows:

```
$ cp -R ch14 ch15
```

2. Open the new **ch15** folder in VS Code.
3. Create a new **backend/src/db/models/message.js** file.
4. Inside it, define a new **messageSchema**, which we are going to use to store chat messages in the database:

```
import mongoose, { Schema } from 'mongoose'
const messageSchema = new Schema({
```

5. The message schema should contain **username** (person who sent the message), **message**, a **room** that it was sent in, and a **sent** date for when the message was sent:

```
  username: { type: String, required: true },
  message: { type: String, required: true },
  room: { type: String, required: true },
  sent: { type: Date, expires: 5 * 60, default: Date.now, required: true },
})
```

For the **sent** date, we specify **expires** to make the messages automatically expire after 5 minutes (**5 * 60** seconds). This ensures that our database does not get cluttered with lots of chat messages. We also set the **default** value to **Date.now** so that all messages are by default tagged as being sent at the current time.

INFO

MongoDB only actually checks for data expiration once a minute, so the expired documents might persist for up to a minute past their defined expiration time.

6. Create a model from the schema and export it:

```
export const Message = mongoose.model('message', messageSchema)
```

After creating the Mongoose schema and model, let's move on to creating the service functions for dealing with chat messages.

Creating the service functions

We need to create service functions to save a new message in the database and to get all messages sent in a given room, sorted by **sent** date, showing the oldest messages first. Follow these steps to implement the service functions:

1. Create a new **backend/src/services/messages.js** file.
2. Inside it, import the **Message** model:

```
import { Message } from '../db/models/message.js'
```

3. Then, define a function to create a new **Message** object in the database:

```
export async function createMessage({ username, message, room }) {  
  const messageDoc = new Message({ username, message, room })  
  return await messageDoc.save()  
}
```

4. Also, define a function to get all messages from a certain room, listing the oldest messages first:

```
export async function getMessagesByRoom(room) {  
  return await Message.find({ room }).sort({ sent: 1 })  
}
```

Next, we are going to use these service functions in our chat server.

Storing and replaying messages

Now that we have all the functions, we need to implement storing and replaying messages in our chat server. Follow these steps to implement the functionality:

1. Edit **backend/src/socket.js** and import the service functions we defined earlier:

```
import { createMessage, getMessagesByRoom } from '../services/messages.js'
```

2. When a new user connects, get all messages from the current room, and send (replay) them to the user using **socket.emit**:

```
export function handleSocket(io) {
  io.on('connection', async (socket) => {
    const room = socket.handshake.query?.room ?? 'public'
    socket.join(room)
    console.log(socket.id, 'joined room:', room)
    const messages = await getMessagesByRoom(room)
    messages.forEach(({ username, message }) =>
      socket.emit('chat.message', { username, message }),
    )
  })
}
```

3. Additionally, when a user sends a message, store it in the database:

```
socket.on('chat.message', (message) => {
  console.log(`${socket.id}: ${message}`)
  io.to(room).emit('chat.message', {
    username: socket.user.username,
    message,
  })
  createMessage({ username: socket.user.username, message, room })
})
```

4. Start the frontend server as follows:

```
$ npm run dev
```

5. Then, start the backend server (do not forget to start the Docker container for the database!):

```
$ cd backend/
$ npm run dev
```

6. Go to **http://localhost:5173/**, log in and send some messages. Then, open a new tab, log in with a different user, and you will see the messages sent earlier get replayed:

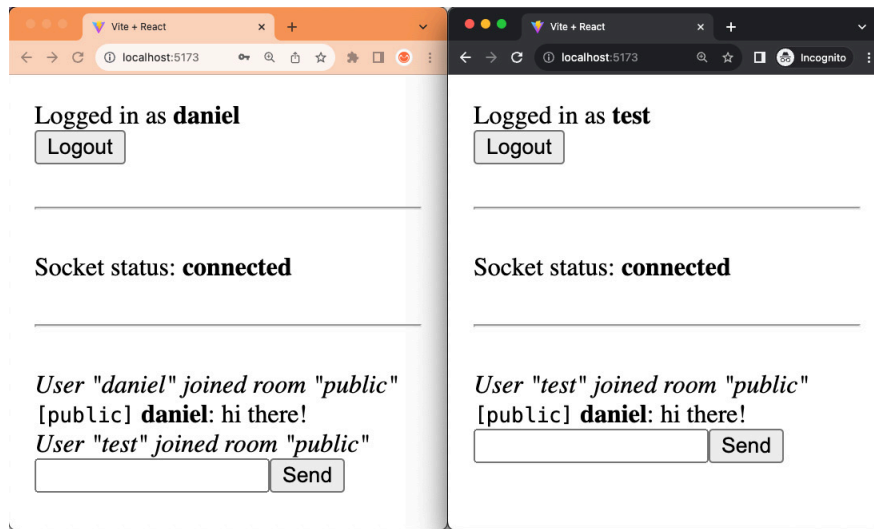


Figure 15.1 – Successfully replaying stored messages

NOTE

The screenshot in Figure 15.1 is from a later version of the app, where we show messages when a user joins a room (we are going to implement these messages later in this chapter). Here, we use these messages to show that replaying works when the user joins after a message was sent.

If you wait 5 minutes and then join the chat again, you will see that the existing messages have expired and are not replayed anymore.

Now, let's make it clearer which messages were replayed in the user interface.

Visually distinguishing replayed messages

Currently, it looks like the other user sent the message right after we joined. It is not clear that the message was replayed from the server. To solve this issue, we can visually distinguish replayed messages by, for example, making them slightly grayer. Let's do that now, as follows:

1. Edit `backend/src/socket.js` and add a `replayed` flag to the replayed messages:

```
const messages = await getMessagesByRoom(room)
messages.forEach(({ username, message }) =>
  socket.emit('chat.message', { username, message, replayed: true }),
)
```

2. Now, edit `src/components/ChatMessage.jsx`, and if the `replayed` flag was set, display the messages with a lower opacity:

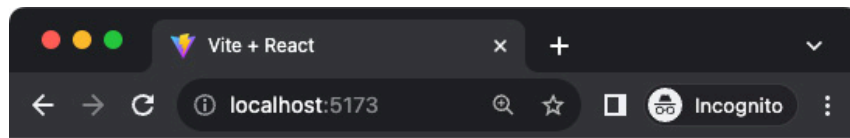
```
export function ChatMessage({ username, message, replayed }) {
```

```
return (
  <div style={{ opacity: replayed ? 0.5 : 1.0 }}>
```

3. Do not forget to update **propTypes** and add the **replayed** flag:

```
ChatMessage.propTypes = {
  username: PropTypes.string,
  message: PropTypes.string.isRequired,
  replayed: PropTypes.bool,
}
```

4. Go to **http://localhost:5173/** again and repeat the same procedure (sending messages from one user, then logging in with a different user in another tab), and you will see that replayed messages are now easy to distinguish from new messages:



Logged in as **test**

Logout

Socket status: **connected**

User "test" joined room "public"

[public] **daniel**: hi there!

[public] **daniel**: new message

Send

Figure 15.2 – Replayed messages are now displayed in a lighter color

Now that we have successfully stored our message history in the database, let's focus a bit on refactoring the chat app to make it more extensible and maintainable in the future.

Refactoring the app to be more extensible

For the refactoring, we will start by defining service functions for all chat functionality that our server provides.

Defining service functions

Follow these steps to get started defining service functions for the chat functionality:

1. Create a new **backend/src/services/chat.js** file.
2. Inside it, import the service functions related to messages:

```
import { createMessage, getMessagesByRoom } from './messages.js'
```

3. Define a new function to send a private message directly to a user:

```
export function sendPrivateMessage(
  socket,
  { username, room, message, replayed },
) {
  socket.emit('chat.message', { username, message, room, replayed })
}
```

Private messages will be used to, for example, replay messages to a specific user, and are not stored in the database.

4. Also, define a function to send a system message:

```
export function sendSystemMessage(io, { room, message }) {
  io.to(room).emit('chat.message', { message, room })
}
```

System messages will be used to, for example, announce that a user joined a room. We also do not want to store these in the database.

5. Then, define a function to send a public message:

```
export function sendPublicMessage(io, { username, room, message }) {
  io.to(room).emit('chat.message', { username, message, room })
  createMessage({ username, message, room })
}
```

Public messages will be used to send regular chat messages to a room. These messages are stored in the database so that we can replay them later.

6. We also define a new function to join a given **socket** to a **room**:

```
export async function joinRoom(io, socket, { room }) {
  socket.join(room)
```

7. Inside this function, send a system message telling everyone in the room that someone joined:

```
    sendSystemMessage(io, {
      room,
      message: `User "${socket.user.username}" joined room "${room}"`,
    })
```

8. Then, replay all messages that were sent to the room privately to the user that just joined it:

```
    const messages = await getMessagesByRoom(room)
    messages.forEach(({ username, message }) =>
      sendPrivateMessage(socket, { username, message, room, replayed: true })
    )
  }
```

9. Lastly, define a service function to get the user info from the **socketId**. We simply copy and paste the code we have previously had in **backend/src/socket.js** here:

```
export async function getUserInfoBySocketId(io, socketId) {
  const sockets = await io.in(socketId).fetchSockets()
  if (sockets.length === 0) return null
  const socket = sockets[0]
  const userInfo = {
    socketId,
    rooms: Array.from(socket.rooms),
    user: socket.user,
  }
  return userInfo
}
```

Now that we have created the service functions for the chat functionality, let's use them in the Socket.IO server.

Refactoring the Socket.IO server to use the service functions

Now that we have defined service functions, let's refactor the chat server code to use them. Follow these steps to do so:

1. Open **backend/src/socket.js** and find the following import:

```
import { createMessage, getMessagesByRoom } from './services/messages.js'
```

Replace the preceding import with the following import to the new chat service functions:

```
import {
  joinRoom,
  sendPublicMessage,
  getUserInfoBySocketId,
} from './services/chat.js'
```

2. Replace the whole **handleSocket** function with the following new code.

When a connection is made, we automatically join the public room using the **joinRoom** service function:

```
export function handleSocket(io) {
  io.on('connection', (socket) => {
    joinRoom(io, socket, { room: 'public' })
  })
}
```

3. Then, define a listener for the **chat.message** event and send it to the given room by using the **sendPublicMessage** service function:

```
socket.on('chat.message', (room, message) =>
  sendPublicMessage(io, { username: socket.user.username, room, message })),
)
```

NOTE

We changed the signature of the **chat.message** event to require passing a room now, so that we can implement a better way to deal with multiple rooms later. Later, we need to make sure to adjust the client code to accommodate this.

4. Next, define a listener for the **user.info** event, in which we use the **async** service function **getUserInfoBySocketId** and return the result of it in **callback** to turn this event into an acknowledgment:

```
socket.on('user.info', async (socketId, callback) =>
  callback(await getUserInfoBySocketId(io, socketId)),
)
})
```

5. Finally, we can re-use the authentication middleware from before:

```
io.use((socket, next) => {
  if (!socket.handshake.auth?.token) {
    return next(new Error('Authentication failed: no token provided'))
  }
})
```

```

    jwt.verify(
      socket.handshake.auth.token,
      process.env.JWT_SECRET,
      async (err, decodedToken) => {
        if (err) {
          return next(new Error('Authentication failed: invalid token'))
        }
        socket.auth = decodedToken
        socket.user = await getUserInfoById(socket.auth.sub)
        return next()
      },
    )
  })
}

```

Now that our chat server is refactored, let's continue with refactoring the client-side code.

Refactoring the client-side code

Now that our server-side code uses service functions to encapsulate the functionality of the chat app, let's do a similar refactoring of the client-side code by extracting client-side commands into separate functions, as follows:

1. Edit **src/hooks/useChat.js** and within the **useChat** hook, define a new function to clear the messages:

```

function clearMessages() {
  setMessages([])
}

```

2. Then, define an **async** function to get all rooms that the user is in:

```

async function getRooms() {
  const userInfo = await socket.emitWithAck('user.info', socket.id)
  const rooms = userInfo.rooms.filter((room) => room !== socket.id)
  return rooms
}

```

3. We can now use these functions in the **sendMessage** function, as follows:

```

async function sendMessage(message) {
  if (message.startsWith('/')) {
    const command = message.substring(1)
    switch (command) {
      case 'clear':
        clearMessages()

```

```

        break
      case 'rooms': {
        const rooms = await getRooms()
        receiveMessage({
          message: `You are in: ${rooms.join(', ')}`,
        })
        break
      }
    }
  }

```

4. Lastly, we adjust the **chat.message** event to send **room** in addition to **message**. For now, we always send messages to the **'public'** room:

```

      default:
        receiveMessage({
          message: `Unknown command: ${command}`,
        })
        break
    }
  } else {
    socket.emit('chat.message', 'public', message)
  }
}

```

In the next section, we will expand this to be able to switch between different rooms.

5. Go to **http://localhost:5173/** and verify that the chat app still works the same way as before.

Now that we have successfully refactored our chat app to be more extensible, let's test out the flexibility of the new structure by implementing new commands to join and switch rooms.

Implementing commands to join and switch rooms

Let's now test out the new structure by implementing commands to join and switch rooms on the chat app, as follows:

1. Edit **backend/src/socket.js** and define a new listener below the **chat.message** listener, which will call the **joinRoom** service function when we receive a **chat.join** event from the client:

```

socket.on('chat.join', (room) => joinRoom(io, socket, { room }))

```

As we can see, having a **joinRoom** service function makes it really simple to reuse the code to join a new room here. It already sends a system message telling everyone that someone joined the room, just like

it does when the user joins the **public** room by default upon connection.

2. Edit `src/components/ChatMessage.jsx` and display **room**:

```
export function ChatMessage({ room, username, message, replayed }) {
  return (
    <div style={{ opacity: replayed ? 0.5 : 1.0 }}>
      {username ? (
        <span>
          <code>[{room}]</code> <b>{username}</b>: {message}
        </span>
      )}
```

3. Add the **room** prop to the **propTypes** definition:

```
ChatMessage.propTypes = {
  username: PropTypes.string,
  message: PropTypes.string.isRequired,
  replayed: PropTypes.bool,
  room: PropTypes.string,
}
```

4. Now, edit `src/hooks/useChat.js` and define a state hook to store the room we are currently in:

```
export function useChat() {
  const { socket } = useSocket()
  const [messages, setMessages] = useState([])
  const [currentRoom, setCurrentRoom] = useState('public')
```

By default, we are in the **public** room.

5. Define a new function to switch rooms:

```
function switchRoom(room) {
  setCurrentRoom(room)
}
```

At the moment, we are only calling **setCurrentRoom** here, but we might want to extend this function later, so it is good practice to abstract it in advance into a separate function.

6. Define a new function to join a room by sending the **chat.join** event and switching the current room:

```
function joinRoom(room) {
  socket.emit('chat.join', room)
  switchRoom(room)
}
```

7. Change the **sendMessage** function to accept arguments for commands, as follows:

```

async function sendMessage(message) {
  if (message.startsWith('/')) {
    const [command, ...args] = message.substring(1).split(' ')
    switch (command) {

```

We can now send commands such as `/join <room-name>` and the room name will be stored in `args[0]`.

8. Define a new command to join a room, in which we first check whether arguments were passed to the command:

```

case 'join': {
  if (args.length === 0) {
    return receiveMessage({
      message: 'Please provide a room name: /join <room>',
    })
  }
}

```

9. Then, we ensure that we have not already joined the room by using the `getRooms` function:

```

const room = args[0]
const rooms = await getRooms()
if (rooms.includes(room)) {
  return receiveMessage({
    message: `You are already in room "${room}"`,
  })
}

```

10. Finally, we can join the room by using the `joinRoom` function:

```

joinRoom(room)
break
}

```

11. Similarly, we can implement the `/switch` command as follows:

```

case 'switch': {
  if (args.length === 0) {
    return receiveMessage({
      message: 'Please provide a room name: /switch <room>',
    })
  }
  const room = args[0]
  const rooms = await getRooms()
  if (!rooms.includes(room)) {
    return receiveMessage({
      message: `You are not in room "${room}". Type "/join ${room}" to join it
    })
  }
}

```

```

    switchRoom(room)
    receiveMessage({
      message: `Switched to room "${room}".`,
    })
    break
  }

```

In this case, we are checking whether the user is in the room already. If not, we tell them that they must join the room first before switching to it.

12. Adjust the `chat.message` event to send to the `currentRoom`, as follows:

```

    } else {
      socket.emit('chat.message', currentRoom, message)
    }

```

13. Go to `http://localhost:5173/`, send a message to the public room, then join the `react` room by executing the `/join react` command. Send a different message to that room.
14. Open another browser window, log in with a different user, and you will see that the first message from the `public` room gets replayed. However, we do not see the message from the `react` room, because we have not joined it yet!
15. Now, in the second browser window, call `/join react` as well. You will see that the second message gets replayed now.
16. Try using `/switch public` to switch back to the `public` room and send another message there. You will see that both clients receive it because they are both in the `public` room.

The result of these actions can be seen in the following screenshot:

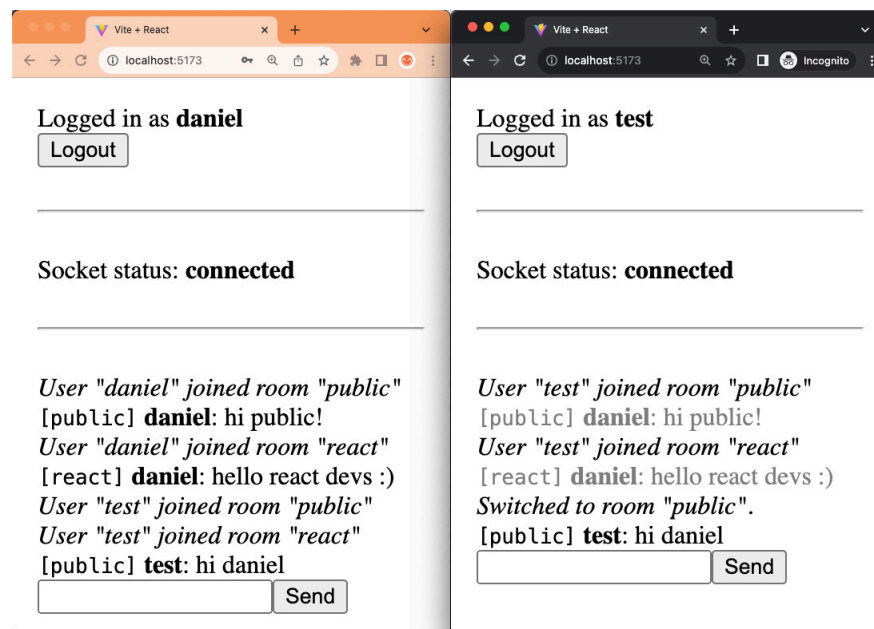


Figure 15.3 – Chatting in different rooms

Summary

In this chapter, we first connected our chat app to the database by storing messages in MongoDB. We also learned how to make documents expire after a certain amount of time. Then, we implemented functionality to replay messages when a new user joins the chat. Next, we spent some time refactoring the chat app to make it more extensible and maintainable in the future. Finally, we implemented ways to join new rooms and switch between rooms.

Up until now, we have only been using libraries to develop our apps. In the next chapter, [**Chapter 16**](#), *Getting Started with Next.js*, we will learn how to use a full-stack React framework for developing apps.

Frameworks, such as Next.js, provide more structure for our apps and offer us a lot of features, such as server-side rendering, out of the box.